



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 4 – Templates and Exception Handling
Topic – 4.2 Class Template, Template and Inheritance

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



CLASS TEMPLATE

- Class Template can also be defined just like Function Template.
- When a class uses the concept of Template, then the class is known as generic class.

- Syntax

```
template<class Ttype>
```

```
class class_name
```

```
{  
  
};
```

- Ttype is a placeholder name which will be determined when the class is instantiated. We can define more than one generic data type using a comma-separated list. The Ttype can be used inside the class body.



CLASS TEMPLATE

- Now, we create an instance of a class using,

`class_name<Ttype> ob;`

where class_name: It is the name of the class.

Ttype: It is the type of the data that the class is operating on.

ob: It is the name of the object.



EXAMPLE OF CLASS TEMPLATE

```
#include <iostream>
using namespace std;
template<class T>
class A
{
    public:
    T num1, num2;
    void add()
    {
        cout<<"Give input for num1 & num2:";
        cin>>num1>>num2;
        cout<<"Addition of num1 & num2 :"  
        <<num1+num2;
    }
};
```

```
int main()
{
    A<int> d;
    d.add();
    return 0;
}
```

Output:

Give input for num1 & num2: 5 6
Addition of num1 & num2 : 11



CLASS TEMPLATE WITH MULTIPLE PARAMETERS

Syntax:

```
template<class T1, class T2, .....>
```

```
class class_name
```

```
{
```

```
    // Body of the class.
```

```
}
```



Example when class template contains two generic data types

```
#include <iostream>
using namespace std;
template<class T1, class T2>
class A
{
    T1 a;
    T2 b;
public:
    A(T1 x,T2 y)
    {
        a = x;
        b = y;
    }
    void display()
    {
```

```
        cout << "Values of a & b: " << a<<" ,"<<b;
```

```
    }
};
int main()
{
    A<int,float> d(5,6.5);
    d.display();
    return 0;
}
```

Output:

Values of a and b: 5, 6.5



Template and Inheritance

It is possible to inherit from a template class. All the usual rules for inheritance and polymorphism apply to class template too.

Use of templates with respect to inheritance involves the followings :

- Derive a class template from a base class, which is a template class.
- Derive a class template from a base class, which is a template class, add more template members in the derived class.
- Derive a class from a base class which is not a template.
- Derive a class from a base class which is a template class and restrict the template feature, so that the derived class and its derivatives do not have the template feature.



Syntax for declaring template derived class from template base class:

```
template <class T1, .....>
```

```
class baseclass
```

```
{
```

```
    // template type data and functions
```

```
};
```

```
template <class T1, .....>
```

```
class derivedclass : public baseclass <T1, ....>
```

```
{
```

```
    // template type data and functions
```

```
};
```




Example of derived class template inherits base class template

```
template<class T>
class Utility
{
public:
    void operation1(const T& val);
    int  operation2();
    bool operation3();
    void operation4(const T& val);
    T    operation5();
    void operation6(const T& val);

private:
    T data;
};

template<class T>
class Adapter: private Utility<T>
{
public:
    void newOperation(const T& val)
    {
        Utility<T>::operation1(val);
    }

    T anotherOperation()
    {
        return Utility<T>::operation5();
    }
};
```

Private inheritance to hide base class operations



Example of deriving from a non-template base class

```
class NonTemplate
{
public:
    void setData(int val);
    int  getData();
    // Other non-template methods...

private:
    int data;
};

template <typename T>
class Derived : public NonTemplate
{
public:
    void set(const T& val);
    T    get();

private:
    T derived_data;
};
```

Non-template type data and operations

template type-specific data and operations

Thank You..